Mission: Please Don't Die

Mars Habitat Automation Platform

Nicola Moscufo (2254216) • Antonio Rubino (2235484)
Sapienza University of Rome

FastAPI • RabbitMQ • SQLite • React • Vite • Docker Compose

Agenda

01

Problem & Scope

Baseline requirements and team objectives

02

Architecture

System components and data flow

03

Rule Engine

Processing logic and persistence

04

Dashboard

Monitoring, control, and real-time updates

05

User Stories

Implementation coverage

06

Live Demo

Step-by-step walkthrough

Problem & Team Scope

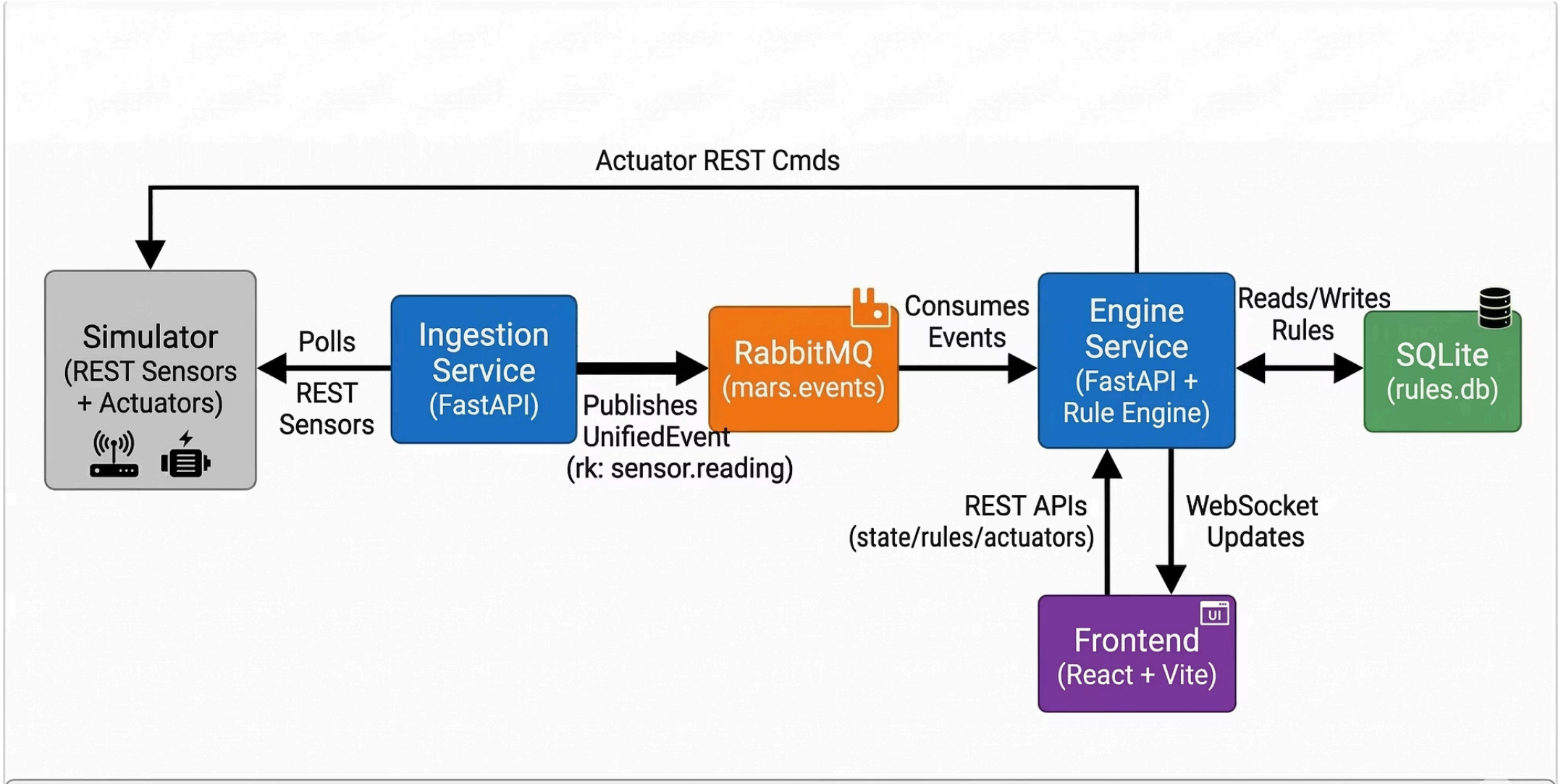
Autonomous Habitat Control

Automated response to environmental conditions ensures crew survival in Mars habitat. Real-time sensor monitoring triggers actuator adjustments without human intervention.

Team Objectives

- REST sensor polling and event normalization
- Broker-based message processing
- Rules engine with conditional logic
- Real-time dashboard for monitoring

System Architecture

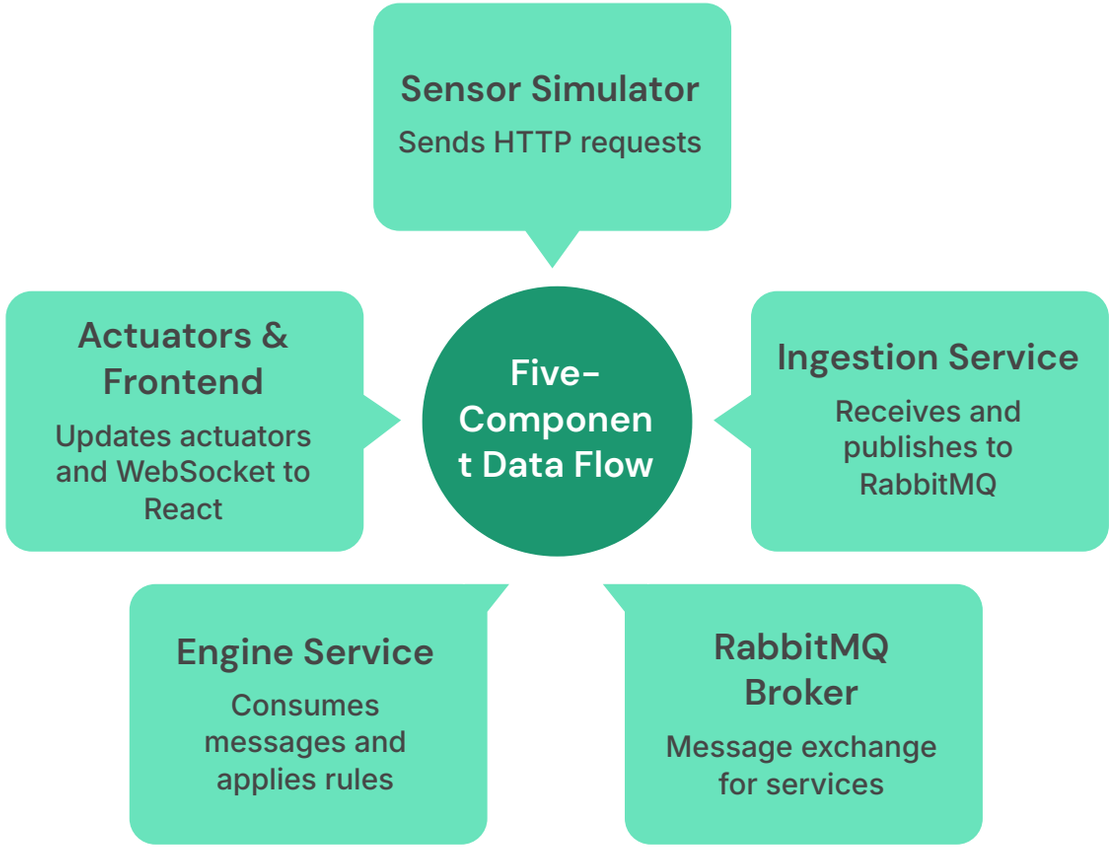


Separation of concerns: ingestion, processing, presentation. Broker-based asynchronous decoupling.

Baseline Requirements Coverage

Requirement	Component	Status
Sensor discovery and values	Ingestion Service	Done
Actuator control interface	Frontend + Engine	Done
Rule creation and management	Engine Service	Done
Real-time dashboard updates	WebSocket + Frontend	Done
Data persistence	SQLite + Docker	Done

System Architecture Overview



Microservices communicate via RabbitMQ exchange. Engine Service maintains state and applies conditional logic. Frontend subscribes to real-time updates.

Containers & Responsibilities

Container	Port	Responsibility
sensor-simulator	8080	Generate mock sensor readings
rabbitmq	5672	Message broker
rabbitmq-web	15672	Management UI
ingestion-service	8001	Poll sensors, normalize events
engine-service	8002	Process rules, control actuators
frontend	3000	Dashboard interface

Unified Event Contract

Event Structure

```
{
  "event_id": "uuid",
  "timestamp": "ISO8601",
  "sensor_name": "string",
  "value": "number",
  "unit": "string",
  "schema_family": "string"
}
```

Routing Details

- **Exchange:** `mars.events`
- **Routing Key:** `sensor.reading`
- JSON schema ensures consistency
- Schema family enables processing

End-to-End Processing Flow

1. **Simulator Generates**
   Sensor readings via HTTP endpoints
2. **Ingestion Polls**
   REST API requests every 5 seconds
3. **Normalize Event**
   Standardize fields into unified format
4. **Broker Publish**
   RabbitMQ exchanges route messages
5. **Engine Consumes**
   Rules evaluated on each event
6. **Actuator Command**
   ON/OFF signals sent to hardware
7. **Dashboard Update**
   WebSocket pushes state changes

Rule Engine & Persistence

Rule Format

```
IF [unit] THEN set to ON|OFF

Operators: <, <=, =, >, >=
```

Persistence

- SQLite database at `/data/rules.db`
- Docker volume `engine_data`
- Rules survive container restart

Actuator Command De-duplication

Engine Service tracks actuator states and suppresses duplicate commands. Prevents excessive hardware cycling and ensures idempotent operations.

Key User Stories (Examples)

ID User Story	Esempio Pratico
US-01	Visualizza i sensori REST disponibili nel sistema.
US-02/03	Controlla valori e timestamp aggiornati dei sensori.
US-05	Configura una regola 'SE umidità > X ALLORA accendi ventola'.
US-07	Attiva o disattiva una regola di automazione esistente.
US-09	Vedi gli aggiornamenti del sensore in tempo reale sulla dashboard.
US-10	Attiva o disattiva manualmente un attuatore tramite interfaccia.

Queste storie sono completamente integrate nel flusso end-to-end: ingest → broker → engine → dashboard.

US-10

Actuator Toggle

Cooling Fan [OFF]

↓ toggle ↓

Backend request:

POST /api/actuators/cooling_fan

```
{"state":"ON"}
```

↓

Cooling Fan [ON]

Updated UI

US-06

Rule List + Edit

#12 Cool greenhouse [Edit]

Rule Form - Edit #12

Name

Cool greenhouse

IF

sensor	operator
greenhouse_temperature	>

threshold	unit
27	C

THEN

actuator	state
cooling_fan	ON

Update Rule #12

Dashboard: Monitoring & Control

Sensor Discovery

Automatically detects available sensors

Latest Values

Current readings with timestamps

Actuator Control

Manual override switches